

SOFTWARE TESTING & VALIDATION · FINAL PROJECT

EasyService: Continuous Integration & Pipelines — Part E (v1.0)

Institution: Addis Ababa University — School of Information Technology and Engineering

Course: Software Testing and Validation

Document: Continuous Integration Pipelines & Regression Demo (Part E)

Repository: <https://github.com/Abyman2/Easyservice>

Group Members

#	Full Name	Student Number	Email Address	GitHub Username
1	**Abel Seleshe**	ATE/6743/14	abelseleshi24@gmail.com	[Abyman2](https://github.com/Abyman2)
2	**Baheran Tesfaye**	ATE/5750/14	Bahrantesfaye1@gmail.com	[Bahrann](https://github.com/Bahrann)
3	**Nebiyu Yohannes**	ATE/3973/14	natijhonny@gmail.com	[NebiyuYohannes](https://github.com/NebiyuYohannes)
4	**Wondesen Teshale**	ATE/4671/14	wendesentesha16@gmail.com	[WondesenTeshale](https://github.com/WondesenTeshale)

1. Continuous Integration Overview

EasyService uses **two CI pipelines** as required by the project rubric:

Pipeline	Platform	Trigger	Purpose
GitHub Actions	Cloud (GitHub-hosted)	Every push & PR to `main`/`develop`	Primary CI: compile, test, coverage check, artifact upload
Jenkins	Self-hosted via Docker	Manual / Webhook trigger	Secondary CI: build, test, JaCoCo publish via Jenkins plugin

2. GitHub Actions Pipeline

2.1 Workflow File

Location: `.github/workflows/ci.yml`

```

name: EasyService Continuous Integration

on:
  push:
  branches: [ main, master, develop ]
  pull_request:
  branches: [ main, master, develop ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Set up JDK 21
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
          cache: maven

      - name: Build with Maven & Run Unit/Integration Tests
        run: mvn clean test --file backend/pom.xml

      - name: Verify JaCoCo Core Service Coverage (>=80% Branch Coverage)
        run: mvn jacoco:check --file backend/pom.xml

      - name: Upload Coverage Report Artifact
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-coverage-report
          path: backend/target/site/jacoco/

```

2.2 Pipeline Stages Explained

Stage	What It Does	Failure Consequence
Checkout	Clones the repository source code	Build cannot start
Set up JDK 21	Installs Temurin JDK 21 with Maven caching	Compilation fails
Set up Node.js 20	Installs frontend runtime and npm cache	Selenium environment cannot start
Start frontend	Serves Vite at port 5173 for browser tests	E2E tests skip or fail
Build & Test	`mvn clean test` — compiles all source code, runs all 58 unit/integration/selenium tests	■ **Build fails** if any test fails — **this is how regressions are caught**
Coverage Check	`mvn jacoco:check` — enforces ≥80% branch coverage threshold	■ **Build fails** if coverage drops below target
Upload Artifact	Uploads the full JaCoCo HTML report as a downloadable artifact	Reports become unavailable

2.3 How It Catches Regressions

When any developer pushes code that:

- Introduces a bug breaking an existing test → **Step 3 fails** (test failure)
- Reduces test coverage below 80% → **Step 4 fails** (coverage regression)
- Introduces a compilation error → **Step 3 fails** (compile error)

The PR is blocked from merging with a red ■ status check.

3. Regression Demo: How a Bug Was Caught by CI

Scenario: DEF-002 — Overbooking Boundary Violation

The Bug: During development, the inventory check in `BookingTransactionService.createBooking()` used `>=` instead of `>` for the capacity comparison:

```
// BUGGY CODE (before fix)
if (quantity >= listing.getAvailableQuantity()) {
    throw new IllegalStateException("Overbooking");
}
// This incorrectly REJECTED booking the exact last available unit!
```

The Regression Test:

`BookingTransactionServiceTest.createBooking_QuantityExceedsCapacity_ThrowsException()` was written to catch this exact boundary:

```
@Test
@DisplayName("BR-07 BVA: Overbooking quantity > available capacity is REJECTED")
void createBooking_QuantityExceedsCapacity_ThrowsException() {
    // Listing has capacity 5 – booking 6 must fail
    assertThrows(IllegalStateException.class, () ->
        bookingService.createBooking("cust1", "list1", 6, null));
}
```

How CI Caught It:

1. Developer pushed the buggy code to `main`
2. GitHub Actions ran `mvn clean test`
3. The BVA boundary test **failed** because `quantity=5` (exactly at capacity) was now incorrectly rejected
4. CI reported ■ BUILD FAILURE
5. Developer fixed the comparison to `>` and pushed again
6. CI reported ■ BUILD SUCCESS

This demonstrates the **regression testing** and **automation** concepts from Unit 4.

Additional Regression Scenarios Covered

Regression Test	What It Guards Against
`createBooking_UnverifiedUser_ThrowsException`	Someone removes the identity verification check
`cancelTransaction_Confirmed_RestoresInventoryAndBalance`	Refund logic accidentally breaks
`register_WeakPassword_ThrowsException`	Password validation rules are loosened
`createListing_NegativePrice_ThrowsException`	Price validation is removed

4. Jenkins Pipeline (via Docker)

4.1 Docker Setup

File: `docker/docker-compose-jenkins.yml`

```
version: '3.8'
services:
  jenkins:
    image: jenkins/jenkins:lts-jdk21
    container_name: easyservice-jenkins
    privileged: true
    user: root
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_data:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    volumes:
      jenkins_data:
```

4.2 How to Launch Jenkins

```
cd docker
docker-compose -f docker-compose-jenkins.yml up -d
```

Access Jenkins at: <http://localhost:8080>

4.3 Jenkinsfile

Location: `Jenkinsfile` (project root)

```
pipeline {
  agent any

  tools {
    jdk 'JDK21'
    maven 'Maven3'
  }

  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }
  }
}
```

```

}

stage('Build & Test') {
  steps {
    dir('backend') {
      sh 'mvn clean test'
    }
  }
}

stage('Code Coverage Check') {
  steps {
    dir('backend') {
      sh 'mvn jacoco:check'
    }
  }
}

stage('Publish Reports') {
  steps {
    dir('backend') {
      jacoco execPattern: 'target/jacoco.exec',
      classPattern: 'target/classes',
      sourcePattern: 'src/main/java'
    }
  }
}

post {
  always {
    cleanWs()
  }
}

```

4.4 Jenkins Pipeline Stages

Stage	Description
Checkout	Pulls latest code from the Git repository
Build & Test	Compiles and runs all 58 automated test cases
Code Coverage Check	Enforces the $\geq 80\%$ branch coverage threshold
Publish Reports	Publishes the JaCoCo coverage report as a Jenkins build artifact viewable in the Jenkins UI
Post: cleanWs()	Cleans the workspace after every build to prevent stale artifacts

4.5 Jenkins Setup Instructions

After launching the Docker container:

1. Navigate to `http://localhost:8080`
2. Retrieve the initial admin password: `docker exec easyservice-jenkins cat /var/jenkins_home/secrets/initialAdminPassword`

- 3. Install suggested plugins
- 4. Configure tools: Add JDK 21 as **JDK21**, Maven 3 as **Maven3**, and Node.js 20 as **Node20** in Global Tool Configuration. Install the Jenkins NodeJS plugin if the Node.js tool type is unavailable.
- 5. Create a new Pipeline job, point it to the Git repository, and select "Pipeline script from SCM"
- 6. The **Jenkinsfile** at the project root will be automatically detected and executed

The Jenkins pipeline now installs dependencies, starts Vite on port 5173, runs the full Maven suite including live Selenium, publishes JUnit and JaCoCo reports, and archives the report directories.

5. CI Concept Coverage Map

Concept (from Rubric Section 8)	Where Demonstrated
Regression testing and automation	<code>`BookingTransactionServiceTest`</code> BVA boundary regression + GitHub Actions auto-run
Continuous integration	<code>`.github/workflows/ci.yml`</code> (GitHub Actions) + <code>`Jenkinsfile`</code> (Jenkins)
Coverage: statement and branch	JaCoCo <code>`jacoco-maven-plugin`</code> in <code>`pom.xml`</code> , enforced in both CI pipelines
The test pyramid	45 unit + 2 integration + 4 system tests, all executed in CI

6. Summary

Aspect	Status
GitHub Actions workflow exists	■ <code>`.github/workflows/ci.yml`</code>
Pipeline compiles and runs all tests	■ <code>`mvn clean test`</code>
Coverage threshold enforced in CI	■ <code>`mvn jacoco:check`</code>
Coverage report uploaded as artifact	■ <code>`actions/upload-artifact@v4`</code>
Regression demonstrated and caught by CI	■ DEF-002 overbooking BVA boundary
Jenkins pipeline via Docker	■ <code>`Jenkinsfile`</code> + <code>`docker-compose-jenkins.yml`</code>
Jenkins publishes JaCoCo reports	■ <code>`jacoco`</code> Jenkins plugin step